

Coffee & queries

Hybrid Search in PostgreSQL

Combining Vector and Full-Text for Real-World Applications

Shayon Sanyal · Principal PostgreSQL Specialist SA · Lead, Agentic AI for Databases

Postgres Summit US 2026 · New York City
September 30 · 10:30–11:20 EDT · Letterpress

START WITH THE CUSTOMER'S REQUEST

**“Something with bergamot.
Keep it under \$20.”**

Words capture explicit intent. Vectors capture related meaning.
Neither gets to change the budget.



ABOUT ME

Shayon Sanyal

Principal PostgreSQL Specialist Solutions Architect

Tech Lead — Agentic AI for Databases

I help teams build on PostgreSQL, from relational applications to retrieval and agent workflows.

Today: readable SQL, visible ranking decisions, and a working coffee shop.

linkedin.com/in/shayonsanyal

Two questions. Two comparisons.

WHY RETRIEVE?

Model only → grounded

Can the answer establish what this shop actually sells, at this price, in stock?

WHY HYBRID?

Keyword · vector · hybrid

Which candidates survive when requests contain exact terms, related concepts, and constraints?

Fluency is not evidence of current facts. A model can also correctly ask for missing data.

Today: compare retrieval in **Lab**, inspect **Catalog**, measure in **Experiments**, converse in **Concierge**.

MEET THE REGULARS

Three people. Three retrieval questions.



Leo

Curious, precise, budget-conscious

Bergamot · at most \$20



Maya

Decisive, comfort-seeking

"Dessert" · a concept, not a label



Yuki

Origin-focused, particular

Coffee from Japan · a hard constraint

Fictional personas · generated portraits · click a persona to read the brief

One row, two search representations

CATALOG FACTS	SEARCH REPRESENTATION	PURPOSE
Name, origin, tasting notes, description	<code>search_document tsvector</code>	Explicit terms, phrases, Boolean logic
The same content, plus roast	<code>embedding vector(384)</code>	Related meaning with cosine distance
Price, stock, roast, origin	Relational columns	Eligibility before ranking candidates

Indexes: GIN on the text document · HNSW with `vector_cosine_ops`.

The demo uses **BAAI/bge-small-en-v1.5**, locally through FastEmbed.

Give important text more weight

```
setweight(to_tsvector('english', name), 'A') ||  
setweight(to_tsvector('english', origin), 'A') ||  
setweight(to_tsvector('english',  
    coffee_flavor_text(flavor_notes)), 'A') ||  
setweight(to_tsvector('english', description), 'B')
```

Stored as a **generated `tsvector` column**. Schema code handles nulls.

Name, origin, and tasting notes carry **A** weight; description carries **B**.

The Catalog shows the actual document and its lexemes.

Retrieve lexical candidates

```
SELECT id,  
       ts_rank_cd(search_document,  
                  websearch_to_tsquery('english', $1), 32) AS score  
FROM beans  
WHERE search_document @@ websearch_to_tsquery('english', $1)  
      AND in_stock > 0 AND price_cents <= $2  
ORDER BY score DESC, id  
LIMIT $3;
```

\$1 request · **\$2** budget in cents · **\$3** candidate depth

“**Bergamot**” can match a catalog term. “**Dessert**” may not.

Retrieve semantic candidates

```
SELECT id, embedding <=> $1::vector AS distance
FROM beans
WHERE embedding IS NOT NULL
      AND in_stock > 0 AND price_cents <= $2
ORDER BY embedding <=> $1::vector
LIMIT $3;
```

\$1 request embedding · same budget · same candidate depth

Keep the **distance operator ascending** in **ORDER BY** for an HNSW path.

The planner may choose a sequential scan for a small catalog.

ELIGIBILITY COMES FIRST

Rank the coffees the customer can actually choose.

Apply the same budget, stock, roast, and origin predicates inside **both candidate branches**, before their limits.

Fuse ranks, not incompatible raw scores

$$\text{RRF}(d) = \sum 1 / (k + \text{rank}_{\text{method}}(d))$$

ILLUSTRATIVE CANDIDATE	KEYWORD RANK	VECTOR RANK	RRF WITH K = 60
Coffee A	1	3	$1/61 + 1/63 = 0.03227$
Coffee B	—	1	$0 + 1/61 = 0.01639$
Coffee C	2	2	$1/62 + 1/62 = 0.03226$

A missing branch contributes **zero**. The demo gives both branches equal weight.

RRF is a full outer join

```
-- Each branch assigns row_number() after its candidate limit.  
SELECT coalesce(s.id, l.id) AS id,  
        coalesce(1.0 / ($1 + s.semantic_rank), 0) +  
        coalesce(1.0 / ($1 + l.lexical_rank), 0) AS rrf_score  
FROM semantic s  
FULL OUTER JOIN lexical l USING (id)  
ORDER BY rrf_score DESC, s.semantic_score DESC NULLS LAST,  
        coalesce(s.id, l.id);
```

`$1` is RRF k. The app's complete CTE query is in `search.py` and Lab → SQL.

One statement combines both rankings and canonical product rows.

Three knobs that change different things

Knob	What it controls	What to watch
Candidate depth	How many rows each retrieval branch can contribute	A missing candidate cannot be recovered by fusion
RRF k	How sharply contributions fall with rank	Smaller k emphasizes the top ranks more
Minimum cosine	An optional semantic acceptance threshold	Can remove useful semantic candidates; lexical ones can remain

Change **one knob at a time** and inspect the same requests.

HNSW's **ef_search** is a separate index-search setting.

LIVE DEMO · SEVEN MINUTES

Meet → compare → inspect → converse

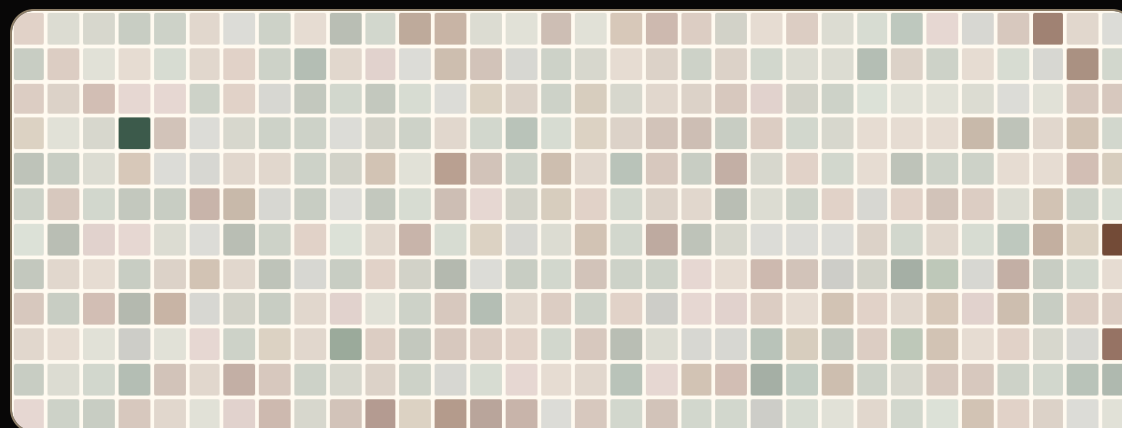
STOP	ACTION	EVIDENCE
Leo	Open brief → Compare this request	Bergamot matches within the \$20 maximum
Maya	Compare “dessert”	Meaning beyond the catalog's exact vocabulary
One coffee	Why this coffee → SQL	Branch ranks and RRF contributions
Yuki	Compare “Coffee from Japan”	Origin eligibility; empty when nothing qualifies
Concierge	Espresso regular → “Cold brew options” → Order that	Same recommended bean; pending approval

localhost:8017 · full script: docs/DEMO.md

From a coffee's text to its vector

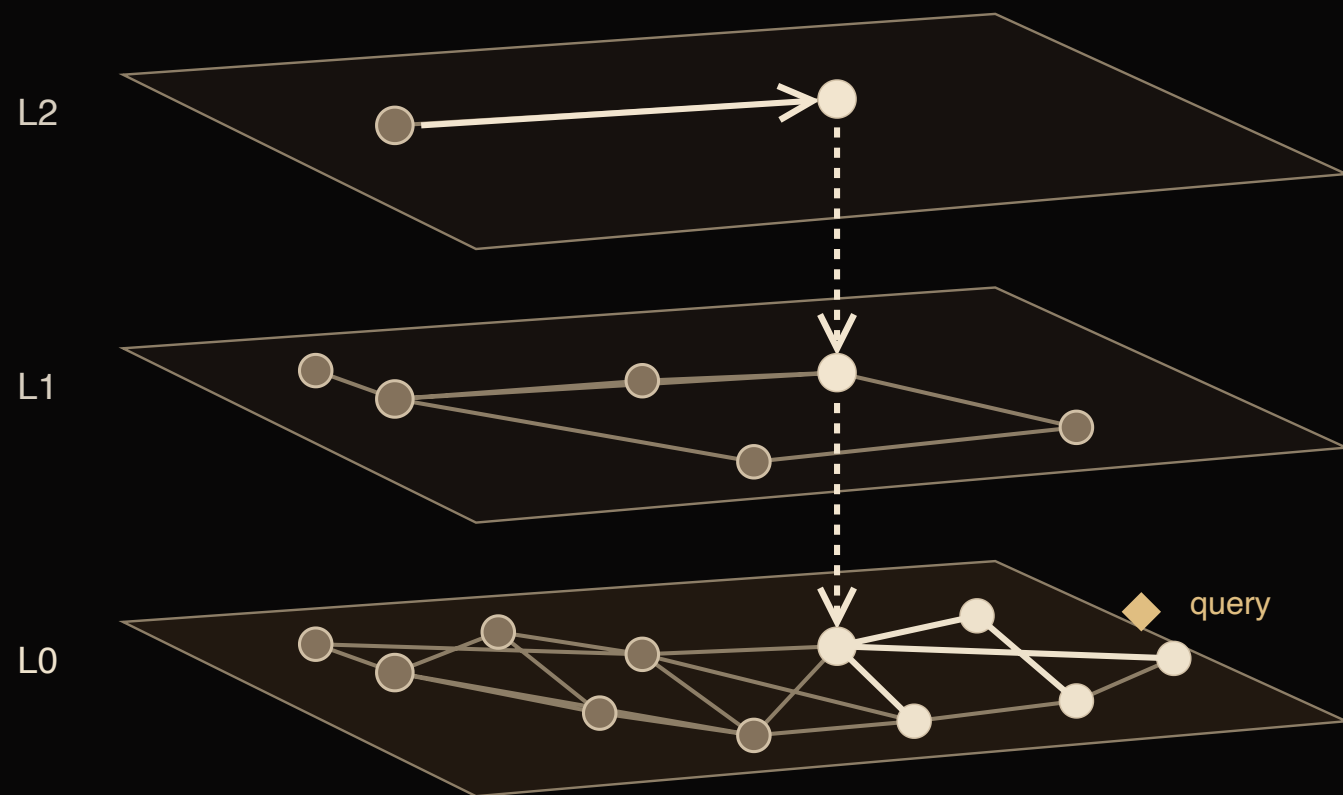
Inspect the representation

1. Open **Catalog** and select a coffee.
2. Compare the source text and weighted lexemes.
3. Inspect the stored **384 dimensions**.
4. Continue below to **How HNSW finds a coffee**.



Actual stored values · dimensions do not have human-assigned flavor labels

HNSW: navigate broadly, then search locally



1. Find a promising region

Greedy moves across sparse links.

2. Descend at the same point

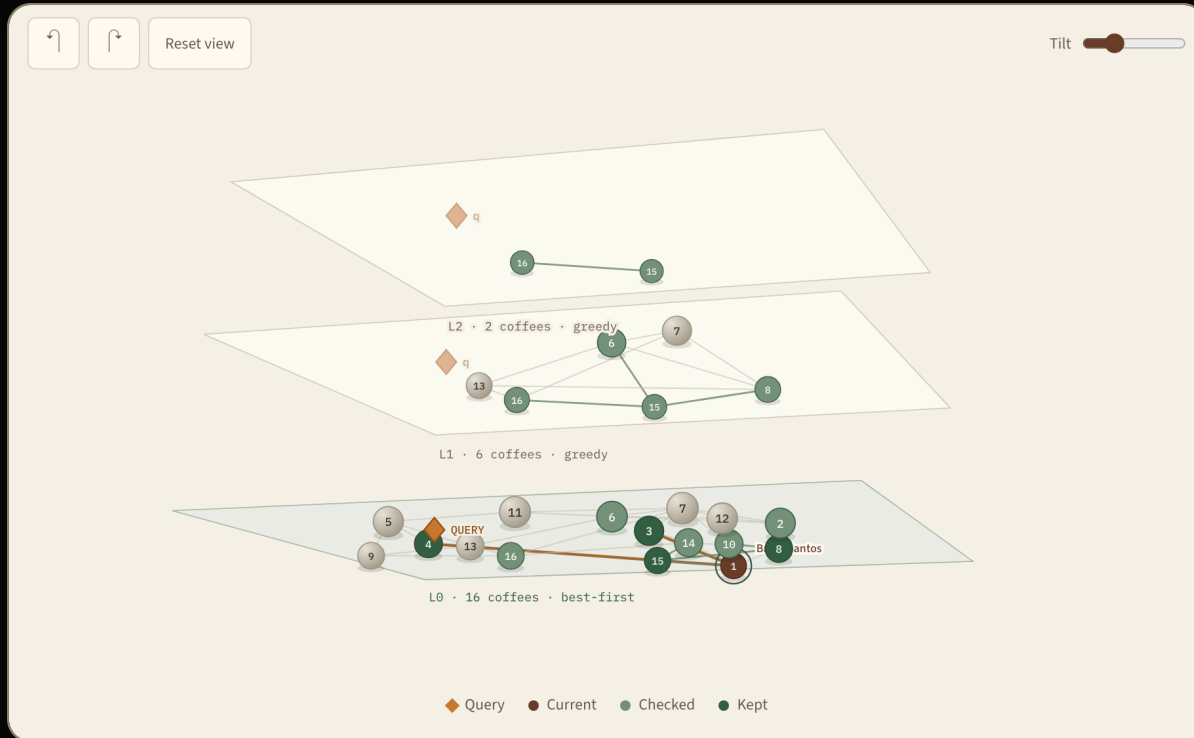
More nodes, more local detail.

3. Explore nearby candidates

A bounded search; ef controls its breadth.

Conceptual hierarchy · the same point may appear at several levels

Follow this query through the graph



Catalog · two-minute walkthrough

Trace query → Next step → descend → inspect candidates → compare with exhaustive neighbors.

Rotate the view. Click a coffee. Change ef and replay.

Real catalog vectors and query distances.
Illustrative layers and links.

Projected layout · no eligibility filters · no database settings changed

A SQL filter is not an ANN pre-filter

An HNSW scan can visit neighbors that the executor later rejects.
A selective filter may leave fewer results than the requested limit.

```
BEGIN;  
SET LOCAL hnsw.ef_search = 100;  
SET LOCAL hnsw.iterative_scan = strict_order; -- pgvector 0.8+  
-- Run the filtered nearest-neighbor query here.  
COMMIT;
```

Consider an exact scan over a selective relational subset, partial indexes, or partitioning when they fit the workload. **Measure the plan and recall.**

Read the plan that actually ran

Lab → SQL → EXPLAIN executes `EXPLAIN (ANALYZE, BUFFERS)`.

- Scan type and index name: did PostgreSQL choose HNSW, GIN, or a table scan?
- Actual rows and filters: where were candidates removed?
- Sorts, buffers, and execution time: where did the work go?
- Parallel workers: two retrieval branches alone do not establish parallelism.

A 16-coffee catalog is good for explanation. It is not a scale benchmark.

Measure the index separately

Experiments → exact versus HNSW uses 12,000 synthetic 384-d vectors in a separate schema, with a real HNSW index.

CHANGE	COMPARE
ef_search	Neighbor recovery against the exact top 20
Selective filter; iterative scan mode	Returned count, recall, actual scan plan
Repeated runs	Query timing and cache/order effects

Recall@20 measures neighbor recovery. It does not measure coffee relevance.

Need an exact term? Make it a constraint.

```
SELECT id, name, embedding <=> $1::vector AS distance
FROM beans
WHERE search_document @@ websearch_to_tsquery('english', $2)
      AND embedding IS NOT NULL
      AND in_stock > 0 AND price_cents <= $3
ORDER BY embedding <=> $1::vector
LIMIT $4;
```

“Related to this question” **AND** “contains the required lexemes.”

This changes the candidate set. RRF's union answers a different requirement.

Evaluate the result, then add complexity

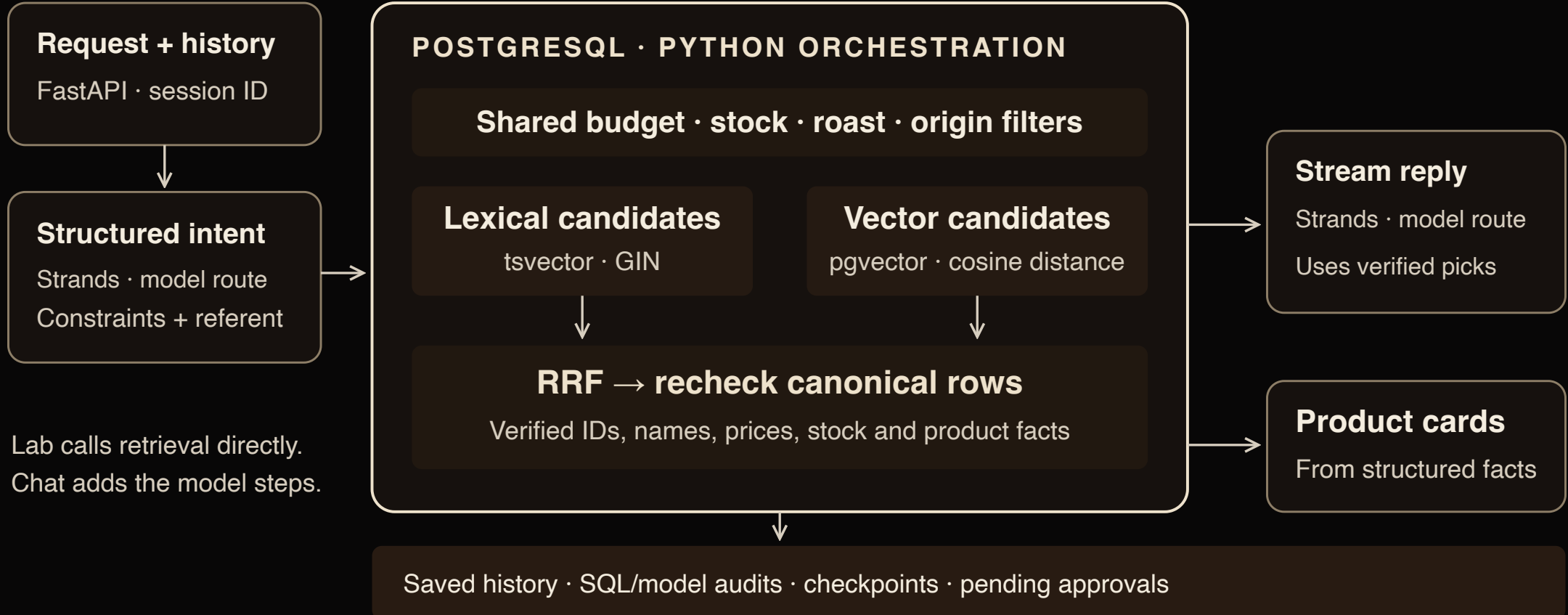
Relevance: judge representative requests, including paraphrases and misses.
Use **Experiments** → **blind judging** to compare without method labels.

Eligibility: independently check budget, origin, roast, and stock.
A plausible answer can still violate a requirement.

Extensions: rerank eligible candidates with preference or popularity signals;
expand an allowed category through a recursive CTE before retrieval.

Keep a baseline. Track whether each addition improves the requests that matter.

Add a conversation around the same retrieval



Model route is selectable · product cards come from canonical catalog rows

“Order that” should refer to the same coffee

1. Complete a recommendation and retain its bean ID in the conversation.
2. Resolve the follow-up against that **same customer and session**.
3. Recheck the selected bean against current facts and constraints.
4. Queue **one pending approval** for that bean and one bag.

The acknowledgment, product card, and approval must agree.

This demo queues a request. It does not charge, fulfill, or decrement stock.

Be precise about what the app proves

INSPECTABLE NOW	STILL REQUIRES IMPLEMENTATION OR EVALUATION
Canonical product cards and eligibility checks	Correctness of every generated sentence
Saved messages and workflow checkpoints	Automatic replay after a crash
SQL and successful model-call audit records	Atomic commit of the entire multi-step turn
Pending approvals	Authorized fulfillment and payment
Local demo with session ownership checks	Authentication, tenant isolation, production load testing

The **data-coverage score** is a **heuristic**, not a probability of correctness.

THREE THINGS TO TAKE HOME

**Retrieve for meaning.
Filter for eligibility.
Measure before you tune.**

Hybrid search is a composable SQL pattern.

The query plan and the returned rows are the evidence.

Run it yourself

```
# From conferences/2026-postgresconf-agentic-ai in the repo:  
python3 -m venv .venv  
source .venv/bin/activate  
python -m pip install -r requirements.txt  
# Configure a dedicated demo database as described in README.  
python scripts/initialize_demo.py  
./run-demo.sh
```

localhost:8017 · Python 3.11+ · PostgreSQL 18 with `vector` and `pg_trgm`

Lab and Catalog use local embeddings. Concierge also needs model access.

github.com/shayons/talks

Thank you.

What does **your** query need to preserve?

Shayon Sanyal · Coffee & queries

github.com/shayons/talks

Appendix • expand a relationship before retrieval

```
WITH RECURSIVE regions(name, parent) AS (  
    VALUES ('Asia-Pacific', NULL::text),  
            ('Indonesia', 'Asia-Pacific'), ('Sumatra', 'Indonesia')  
) , allowed(name) AS (  
    SELECT name FROM regions WHERE name = $1  
    UNION  
    SELECT r.name FROM regions r JOIN allowed a ON r.parent = a.name  
)  
SELECT b.id, b.name  
FROM beans b  
WHERE b.in_stock > 0 AND b.embedding IS NOT NULL  
    AND EXISTS (SELECT 1 FROM allowed a  
                WHERE strpos(lower(b.origin), lower(a.name)) > 0)  
ORDER BY b.embedding <=> $2::vector LIMIT $3;
```

Illustrative taxonomy. Production: canonical relationships and explicit access rules.

Appendix • operate the measured workload

AREA	INSPECT BEFORE CHANGING IT
HNSW build	m, ef_construction, index size, build memory and insert cost
HNSW search	ef_search, filtered recall, iterative scan limits
Text retrieval	Document weights, language configuration, GIN and update cost
Connections	Concurrent database work, pool waits, transaction duration
Retention	Message/audit growth, ANALYZE, vacuum and retention policy

Separate embedding, SQL, and model-call timings. Benchmark representative data, filters, concurrency, and warm/cold conditions before setting service targets.

References and reproducible paths

- **PostgreSQL 18:** Full-text search, WITH and recursive queries, Using EXPLAIN
- **pgvector 0.8.2:** HNSW, filtering, iterative scans, and recall
- **RRF:** Cormack, Clarke & Büttcher, SIGIR 2009
- **HNSW:** Malkov & Yashunin, Hierarchical navigable small world graphs
- **This repository:** `schema.sql`, `search.py`, `static/hnsw-core.mjs`, `experiments.py`
- **Presenter material:** `docs/DEMO.md`, `deck/TALKING_POINTS.md`, `deck/SQL_PATTERNS.md`

Deck aligned with the local application · September 10, 2026